BASES DE DATOS · INGENIERÍA EN SISTEMAS · 3.° AÑO

Trabajo Práctico Integrador

Sistema de gestión y distribución de una empresa vitivinícola: «Bodegas del Atuel»

Dato	Detalle
Cátedra / Facultad	UTN Facultad Regional San Rafael
Ciclo lectivo	2026
Modalidad	Grupal, de 3 a 4 integrantes, con roles rotativos
Motor de base de datos	PostgreSQL (versión 14 o superior)
Acceso a datos desde la API	SQL directo con driver nativo. No se permite ORM
Fecha de lanzamiento	13/10/2026
Entrega final y defensa oral	A definir

Contenido

- 1. Presentación y objetivos
- 2. Modalidad de trabajo y cronograma
- 3. Escenario propuesto
- 4. Reglas de negocio
- 5. Modelo de datos: entidades mínimas
- 6. Tecnologías y restricciones
- 7. Etapas del proyecto (1 a 9), cada una con pasos, entregables y criterios de aceptación
- 8. Consultas SQL requeridas
- 9. Rúbrica de evaluación
- 10. Estructura del repositorio y entrega final
- Anexos: ejemplos en PostgreSQL, catálogos sugeridos, preguntas típicas de defensa

1. Presentación y objetivos

Este trabajo práctico integra todo lo visto en la materia en un único proyecto: se parte de un escenario real, se modela la base de datos, se implementa con PostgreSQL, se carga con un volumen de datos importante, se protege con reglas de integridad escritas en la propia base, se optimiza con índices y finalmente se conecta con una API y un front que se publican en un servidor en la nube.

La idea central es que la base de datos deje de ser un ejercicio aislado y pase a ser el corazón de un sistema que otros programas consumen. Por eso **la parte que más pesa en la evaluación es la capa de datos**: modelo, constraints, índices, funciones, procedimientos, triggers y consultas.

1.1 Objetivos de aprendizaje

- Modelar un escenario real con un diagrama entidad-relación (incluyendo jerarquías, entidades débiles y relaciones N:M con atributos) y transformarlo a un esquema relacional normalizado hasta 3FN.
- Implementar el esquema físico en PostgreSQL aplicando **constraints** (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, DEFAULT), tipos enumerados y dominios.
- Escribir **funciones, procedimientos almacenados y triggers** en PL/pgSQL para garantizar reglas de negocio dentro de la base de datos.
- Cargar masivamente datos coherentes (más de 100.000 registros en al menos una tabla) con scripts reproducibles.
- Diseñar **índices** con evidencia: interpretar planes de ejecución con `EXPLAIN (ANALYZE, BUFFERS)` y medir mejoras.
- Resolver consultas SQL básicas y avanzadas (subconsultas, CTE, funciones de ventana, división relacional, agregaciones por período) y operaciones de inserción, modificación y borrado.
- Manejar **transacciones y concurrencia** y entender qué garantiza la base de datos frente a dos usuarios simultáneos.
- Conectar la base de datos con una **API REST** y un **front** sin ocultar el SQL, y traducir los errores de la base a respuestas HTTP claras.
- Desplegar el sistema en la **nube** con criterios básicos de seguridad: roles con privilegios mínimos, secretos fuera del repositorio, conexión cifrada, backup y restauración probados.

1.2 Qué se evalúa y qué no

- **Se evalúa** la calidad del diseño, la corrección de las reglas implementadas en la base, la evidencia de rendimiento, el dominio de SQL y la capacidad de explicar y defender cada decisión.
- **La API y el front son el medio** para demostrar que la base funciona como parte de un sistema. No se pide un diseño visual sofisticado ni autenticación compleja (puede sumarse como extra).
- **No se permite ORM ni generadores que oculten el SQL.** El esquema, las migraciones y todas las consultas las escriben los alumnos a mano.

2. Modalidad de trabajo y cronograma

2.1 Organización del grupo

- Grupos de **3 a 4 integrantes**. Se sugieren cuatro roles que **rotan por etapa**: Datos (modelo y SQL), API, Front/Integración e Infraestructura (Docker, nube, backups).
- Aunque haya roles, **todos los integrantes deben poder explicar el modelo y las reglas de la base de datos** en la defensa. El rol organiza el trabajo, no limita la responsabilidad.
- Repositorio **Git** obligatorio (GitHub o GitLab) con commits de todos los integrantes. Se recomienda una rama por etapa y revisión cruzada antes de integrar.
- Cada etapa cierra con un entregable parcial en el repositorio (ver cronograma). El docente puede devolver correcciones antes de la siguiente etapa.

2.2 Cronograma sugerido

Hito	Etapa	Entregable
H1	1. Modelado	DER, esquema relacional, diccionario de datos y DDL base
H2	2. Integridad y lógica en la base	Constraints, funciones, procedimientos, triggers, vistas, roles y pruebas SQL
H3	3. Carga masiva	Scripts de carga reproducibles y reporte de conteos y tiempos
H4	4. Índices y rendimiento	Informe con planes de ejecución antes y después
H5	5. Consultas SQL	Archivos de consultas INSERT, UPDATE, DELETE y SELECT comentadas
H6	6. API	API REST funcionando en local con Docker Compose
H7	7. Front	Front conectado a la API
H8	8. Nube	Sistema desplegado con URL pública, backup y restauración probados
H9	9. Documentación y defensa	Informe final, README, demo en vivo y defensa oral

> **Integridad académica.** Todo el trabajo debe ser del grupo.
> En cualquier caso, **en la defensa oral cada integrante debe poder explicar y modificar en vivo cualquier parte del proyecto**. La nota es individual en función de esa defensa.

3. Escenario propuesto

Se desea mantener una base de datos para la gestión de **«Bodegas del Atuel»**, una empresa vitivinícola ficticia que integra la producción de uva y de vino con su distribución y venta a través de una red de salas de venta distribuidas en distintas localidades.

3.1 Localidades, salas de venta y empleados

De cada localidad se conoce su nombre, su provincia y su código postal. Por cada localidad existe un único encargado regional (responsable comercial); es decir, en las localidades con más de una sala de venta, el mismo encargado está afectado a todas las salas de esa localidad. De cada sala de venta se conoce su ID, su dirección (calle y número) y el día de cierre semanal. Una sala está ubicada en una sola localidad, pero en una localidad puede haber más de una sala.

En cada sala trabajan varios empleados. De cada empleado se conoce su CUIT, su nombre, su fecha de ingreso laboral y su salario. Cada empleado trabaja en una sola sala.

3.2 Producción de uva

La empresa cultiva uva en fincas. De cada finca se conoce su ID, su nombre, su superficie total en hectáreas y el departamento donde se ubica (nombre del departamento y provincia). Cada finca se divide en parcelas, identificadas por un número dentro de la finca: una parcela no existe sin su finca. De cada parcela se conoce su superficie, el año de plantación y la cepa que tiene plantada (una sola). De cada cepa se conoce su nombre y su tipo (tinta o blanca).

En cada campaña (año de cosecha), cada parcela puede cosecharse a lo sumo una vez, lo que genera un lote de uva. De cada lote se registra su número, la fecha de cosecha, los kilos cosechados y la bodega a la que fue entregado. A cada lote de uva se le practican uno o más análisis de calidad, de los que se conoce la fecha, el grado Brix, el pH y la acidez total (g/L).

3.3 Bodegas y vinos

Las uvas se elaboran en las bodegas del grupo. De cada bodega se conoce su CUIT, su razón social y su domicilio. Una bodega elabora varios vinos.

Cada vino se identifica por un código y se conoce su nombre comercial, su tipo (tinto, blanco, rosado o espumante), su añada, su presentación (volumen en ml, por ejemplo 375, 750 o 1500), su graduación alcohólica y su precio de venta, que es el mismo para todas las salas. Un vino puede estar compuesto por una o más cepas, cada una en un determinado porcentaje de corte (la suma de los porcentajes debe ser 100). Además, cada vino tiene uno o más maridajes recomendados. De cada maridaje se conoce el nombre (por ejemplo «carnes rojas») y la temperatura de servicio sugerida en °C; un maridaje puede repetirse para distintos vinos.

Por ejemplo, el vino **«Malbec Reserva»**, añada 2021, presentación de 750 ml, cuesta $12.500 y tiene 14,00° de graduación. Está compuesto por Malbec (85 %) y Cabernet Sauvignon (15 %), lo elabora una de las bodegas del grupo y se recomienda con carnes rojas (17 °C) y quesos duros (16 °C).

El sistema deberá permitir consultar alternativas como: vinos de una misma cepa, vinos de una bodega, vinos con el mismo nombre y distinta añada o presentación, entre otras.

3.4 Clientes y ventas

Los clientes compran en las salas de venta. De cada cliente se guarda su ID, apellido y nombre o razón social, tipo y número de documento, dirección, localidad y fecha de alta. Existen tres tipos de cliente:

- **Socios del club** (consumidores finales): reciben un 10 % de descuento.
- **Distribuidores**: reciben un 20 % de descuento. Se guarda su zona de cobertura y su número de habilitación.
- **Exportadores**: reciben un 30 % de descuento. Se guarda el país de destino principal y sus permisos de exportación. Un exportador puede tener varios permisos; de cada uno se conoce el código, la fecha de emisión y la fecha de vencimiento.

De cada venta se genera un comprobante que contiene: número, fecha, cliente, sala en la que se realizó, subtotal, descuento y total. Cada comprobante incluye un conjunto de vinos con los siguientes datos: código de vino, cantidad, precio unitario al momento de la venta, descuento aplicado (directamente relacionado al tipo de cliente) y subtotal del renglón.

3.5 Stock, depósito central, ingresos y transferencias

Para poder efectivizar una venta, cada sala posee su stock de cada vino. La empresa cuenta con un depósito central que recibe toda la producción embotellada y la redistribuye a las salas que la soliciten. El depósito tiene su propio stock por vino.

La forma de ingresar vinos al stock del depósito central es a través de los **INGRESOS**. De cada ingreso se registra: código de ingreso, fecha, estado, transportista que efectuó el traslado y bodega de origen. Cada ingreso contiene un conjunto de vinos con su cantidad de botellas. Cada ingreso tiene un estado (en confección, terminado, procesado): sólo un ingreso terminado puede impactar en el stock del depósito, y una vez impactado pasa a procesado.

Existe una lista de empresas de transporte a las que se puede designar el traslado. De cada una se conoce: código de transporte, razón social, CUIT, e-mail, teléfono y domicilio. Además, se debe conocer a qué localidades alcanza el servicio de cada transportista.

Cada sala que necesite vinos los debe requerir al depósito central por medio de **TRANSFERENCIAS**, que contienen: número de solicitud, sala solicitante y fecha de confección (es la fecha actual y no puede modificarse). Cada transferencia contiene un conjunto de vinos con su cantidad y tiene un estado (en confección, terminado, procesado). Sólo una transferencia terminada puede impactar en el stock: al procesarla se descuenta del depósito central y se suma al stock de la sala. No puede procesarse si el depósito no tiene stock suficiente.

4. Reglas de negocio

El escenario anterior se resume en las siguientes reglas. **Cada regla debe estar protegida por al menos un mecanismo en la base de datos** (constraint, función, procedimiento o trigger), y el grupo debe poder señalar cuál. En el archivo de pruebas de la etapa 2 debe existir al menos un caso por regla.

N.°	Regla	Mecanismo esperado
RN-01	El precio de un vino es único para toda la red de salas y siempre mayor que cero.	Atributo en vino + CHECK
RN-02	Una sala pertenece a una sola localidad. Cada localidad tiene un único encargado regional.	FK + diseño normalizado (3FN)
RN-03	Cada empleado trabaja en una sola sala.	FK NOT NULL
RN-04	Una parcela tiene una sola cepa y se cosecha como máximo una vez por campaña.	FK + UNIQUE (parcela, campaña)
RN-05	Los porcentajes de corte de un vino suman exactamente 100.	Constraint trigger diferido
RN-06	El descuento depende del tipo de cliente: socio 10 %, distribuidor 20 %, exportador 30 %.	Función de descuento + CHECK de rango
RN-07	Un exportador puede tener varios permisos; el vencimiento es posterior a la emisión.	Tabla de permisos + CHECK entre columnas
RN-08	El total del comprobante es la suma de sus renglones con el descuento aplicado. No se carga a mano.	Función de cálculo + trigger
RN-09	No se puede vender más de lo que hay en stock de la sala. El stock nunca es negativo.	Trigger BEFORE INSERT + CHECK (cantidad >= 0)
RN-10	Sólo un ingreso terminado impacta en el depósito y pasa a procesado. Las transiciones son en_confeccion → terminado → procesado y no retroceden.	Tipo ENUM + trigger de estados + procedimiento
RN-11	Igual que RN-10 para transferencias. La fecha de confección es la actual y no se puede modificar.	Trigger de estados y de fecha + procedimiento
RN-12	Un ingreso o una transferencia que no está en confección no admite agregar, modificar ni quitar renglones.	Trigger sobre los renglones
RN-13	Todo cambio de precio de un vino queda auditado (fecha, usuario, precio anterior y nuevo).	Trigger AFTER UPDATE + tabla de auditoría
RN-14	No se puede borrar un transportista, una bodega ni un vino que tengan movimientos asociados.	FK con ON DELETE RESTRICT
RN-15	Cantidades, superficies, kilos y precios son siempre positivos. La graduación está entre 5 y 20.	CHECK

5. Modelo de datos: entidades mínimas

La siguiente tabla lista las entidades que el modelo debe contemplar como mínimo. **Los atributos y las claves son una guía**: el grupo puede agregar los que considere necesarios y debe justificar sus decisiones.

Entidad	Atributos principales	Observaciones
Localidad	Código postal, nombre, provincia, encargado regional	El encargado depende de la localidad
Sala de venta	ID, calle, número, día de cierre, localidad	
Empleado	CUIT, nombre, fecha de ingreso, salario, sala	
Finca	ID, nombre, superficie (ha), departamento, provincia	
Parcela	N.° de parcela, finca, superficie, año de plantación, cepa	Entidad débil: clave (finca, n.° de parcela)
Cepa	ID, nombre, tipo (tinta o blanca)	Catálogo real
Lote de uva	N.° de lote, parcela, campaña, fecha, kg, bodega destino	Una cosecha por parcela y campaña
Análisis de calidad	ID, lote, fecha, Brix, pH, acidez	Varios análisis por lote
Bodega	CUIT, razón social, domicilio	
Vino	Código, nombre, tipo, añada, volumen (ml), graduación, precio, bodega	Un vino lo elabora una bodega
Composición (vino-cepa)	Vino, cepa, porcentaje	N:M con atributo; suma 100 por vino
Maridaje	ID, nombre, temperatura de servicio	Catálogo
Vino-maridaje	Vino, maridaje	N:M
Cliente	ID, nombre o razón social, tipo y n.° de documento, dirección, localidad, fecha de alta, descuento	Supertipo de una jerarquía con 3 subtipos
Socio / Distribuidor / Exportador	Datos propios de cada subtipo (zona y habilitación; país de destino)	Decidir y justificar la estrategia de mapeo
Permiso de exportación	Código, exportador, fecha de emisión, fecha de vencimiento	Varios por exportador
Comprobante	Número, fecha, cliente, sala, subtotal, descuento, total	
Renglón de comprobante	Comprobante, vino, cantidad, precio unitario, descuento, subtotal	Clave (comprobante, vino)
Transportista	Código, razón social, CUIT, e-mail, teléfono, domicilio	
Transportista cubre localidad	Transportista, localidad	N:M
Ingreso / Renglón de ingreso	Código, fecha, estado, transportista, bodega; vino y cantidad	Estado con tipo ENUM

vino incluye comprobante

Entidad	Atributos principales	Observaciones
Transferencia / Renglón de transferencia	Número, sala, fecha de confección, estado; vino y cantidad	Estado con tipo ENUM
Stock de sala	Sala, vino, cantidad	Clave (sala, vino); cantidad >= 0
Stock de depósito	Vino, cantidad	Un solo depósito central; cantidad >= 0
Auditoría de precio	ID, vino, precio anterior, precio nuevo, fecha y hora, usuario	La completa un trigger

Decisiones de diseño que deben tomar y justificar por escrito

- **Jerarquía de clientes:** tabla única con columna de tipo, una tabla por subtipo, o supertipo más subtipos. Indicar ventajas, desventajas y por qué eligieron esa.
- **Claves naturales o subrogadas** (CUIT y código postal frente a identificadores autogenerados). Justificar caso por caso.
- **Tipos de datos:** dinero con `NUMERIC(p,s)`, nunca con `FLOAT`; fechas con `DATE` o `TIMESTAMP`; estados con tipo enumerado.
- **Atributos derivados** (subtotales, totales, descuento): cuáles se almacenan, cuáles se calculan y cómo se garantiza su consistencia.
- **Acciones referenciales:** qué `ON DELETE` y `ON UPDATE` lleva cada clave foránea y por qué (por ejemplo, RESTRICT en movimientos y CASCADE solo en renglones que no existen sin su cabecera).

6. Tecnologías y restricciones

Componente	Requisito	Observaciones
Motor de base de datos	PostgreSQL 14 o superior	Local en contenedor y luego en la nube
Acceso a datos	Driver nativo (por ejemplo `psycopg` en Python, `pg` en Node, JDBC en Java)	**Prohibido ORM** y query builders que oculten el SQL
API	REST con JSON, en el lenguaje que elija el grupo	Documentada con OpenAPI/Swagger o una colección Postman
Front	Libre: HTML/CSS/JS, React, Vue, etc.	Consume únicamente la API
Contenedores	Docker y Docker Compose	Un solo comando debe levantar base, API y front en local
Control de versiones	Git en GitHub o GitLab	Commits de todos los integrantes
Generación de datos	Python con Faker y `random` (u otra herramienta equivalente) más catálogos reales	Semilla fija para que sea reproducible
Modelado	draw.io, dbdiagram.io, pgModeler u otra herramienta	Entregar imagen/PDF y el archivo fuente
Cliente SQL	`psql`, DBeaver o pgAdmin	Las capturas del informe pueden venir de cualquiera

> **Regla de oro.** La API **no recalcula** totales, descuentos ni stock: llama a las funciones y procedimientos de la base y deja que los triggers protejan las reglas. Así las reglas siguen valiendo aunque alguien acceda a la base directamente.

7. Etapas del proyecto

Cada etapa tiene un **objetivo**, una lista de **pasos** a seguir en orden, los **entregables** y los **criterios de aceptación** (lo que el docente va a verificar). Una etapa se considera terminada cuando se cumplen todos sus criterios.

Etapa 1. Modelado

Objetivo: pasar del escenario escrito a un esquema físico en PostgreSQL, pasando por el DER, el esquema relacional y la normalización.

Pasos

1. Leer el escenario (sección 3) y las reglas de negocio (sección 4). Subrayar entidades, atributos, relaciones y cardinalidades, y marcar las reglas que van a requerir lógica adicional.
2. Elaborar el **DER** con una notación consistente: entidades, atributos, claves, cardinalidades mínima y máxima, la entidad débil Parcela, la jerarquía de Cliente y las relaciones N:M con atributos (composición, renglón de comprobante, etc.).
3. Transformar el DER a un **esquema relacional**: una relación por entidad y por relación N:M, con claves primarias y foráneas. Decidir y justificar cómo se mapea la jerarquía de clientes.
4. Verificar la **normalización hasta 3FN**: listar las dependencias funcionales relevantes (por ejemplo, localidad → encargado regional) y mostrar que no quedan dependencias parciales ni transitivas.
5. Armar el **diccionario de datos**: para cada tabla y columna indicar tipo, si admite nulos, restricciones y una descripción.
6. Escribir el **esquema físico** en SQL (`001_esquema.sql`): tipos enumerados, dominios, tablas, claves primarias y foráneas. Las restricciones adicionales se completan en la etapa 2.
7. Verificar que la base se crea **desde cero y sin errores** en un contenedor de PostgreSQL.

Entregables

- DER (PDF o imagen) y archivo fuente.
- Esquema relacional escrito.
- Diccionario de datos.
- Justificación de normalización y de las decisiones de diseño de la sección 5.
- `db/migrations/001_esquema.sql`.

Criterios de aceptación

- ☐ Todas las entidades de la sección 5 están representadas y sus cardinalidades son coherentes con el escenario.
- ☐ El esquema relacional está en 3FN y hay justificación escrita.
- ☐ Ningún importe usa `FLOAT`; los estados no son texto libre.
- ☐ `001_esquema.sql` corre de punta a punta sobre una base vacía.
- ☐ El diccionario de datos coincide con el esquema físico.

Etapa 2. Integridad y lógica en la base de datos

Objetivo: que las reglas de negocio vivan dentro de la base de datos y se cumplan sin importar quién la use. Esta es la etapa de mayor peso de la evaluación.

Requisitos mínimos

Elemento	Mínimo exigido	Ejemplos en este escenario
PRIMARY KEY	En todas las tablas	Claves compuestas en renglones y stock
FOREIGN KEY	Todas las relaciones, con ON DELETE y ON UPDATE elegidos y justificados	RESTRICT en vino, bodega y transportista; CASCADE en renglones
NOT NULL	En todo atributo obligatorio	Fecha, cantidad, precio, estado
UNIQUE	4 restricciones adicionales a las claves primarias	(nombre, añada, volumen) en vino; (parcela, campaña) en lote; CUIT de bodega
CHECK	8 restricciones, incluyendo al menos una entre columnas	cantidad > 0; precio > 0; graduación entre 5 y 20; fecha de vencimiento > fecha de emisión; stock >= 0
DEFAULT	3 valores por defecto	Estado inicial en_confeccion; fecha de alta actual
Tipos propios	1 tipo ENUM y 1 DOMAIN	ENUM de estado de documento; DOMAIN de CUIT de 11 dígitos
Funciones	4 funciones	`fn_total_comprobante`, `fn_stock_sala`, `fn_descuento_cliente`, `fn_antiguedad_anios`
Procedimientos	2 procedimientos transaccionales	`sp_procesar_ingreso`, `sp_procesar_transferencia`
Triggers	6 triggers con propósitos distintos: al menos uno BEFORE, uno AFTER y un constraint trigger diferido	Ver lista de la sección siguiente
Vistas	3 vistas (una puede ser materializada)	Stock total por vino, ventas mensuales por sala, documentos pendientes
Roles y privilegios	3 roles: propietario, lectura/escritura de la API y solo lectura para reportes	REVOKE de privilegios públicos y GRANT mínimos

Triggers mínimos sugeridos

Trigger	Qué garantiza	Regla
Validación de stock en venta (BEFORE INSERT en renglón de comprobante)	Rechaza la venta si no hay stock suficiente en la sala	RN-09
Descuento de stock y cálculo de totales (AFTER INSERT en renglón de comprobante)	Actualiza el stock de la sala y los importes del comprobante	RN-08, RN-09
Control de estados de ingreso y de transferencia	Solo permite en_confeccion → terminado → procesado, sin retrocesos	RN-10, RN-11

Trigger	Qué garantiza	Regla
Bloqueo de renglones (BEFORE INSERT/UPDATE/DELETE en renglones de ingreso y transferencia)	No admite cambios si el documento no está en confección	RN-12
Fecha de confección de transferencia (BEFORE INSERT/UPDATE)	Fuerza la fecha actual al insertar y prohíbe modificarla	RN-11
Auditoría de precio (AFTER UPDATE en vino)	Registra precio anterior, precio nuevo, fecha y usuario	RN-13
Suma de porcentajes de corte (CONSTRAINT TRIGGER DEFERRABLE INITIALLY DEFERRED)	Al cerrar la transacción, la composición de cada vino suma 100	RN-05

¿Dónde va cada regla?

Una parte importante del aprendizaje es decidir qué herramienta corresponde a cada tipo de regla. Esta guía debe aparecer justificada en el informe final.

Tipo de regla	Herramienta	Ejemplo
Valor válido de una sola fila, calculable siempre igual	NOT NULL, CHECK, DOMAIN	precio > 0
Existencia de un dato relacionado	FOREIGN KEY	Un renglón debe apuntar a un vino existente
Unicidad	UNIQUE o PRIMARY KEY	Una cosecha por parcela y campaña
Depende de otras filas, de otras tablas o de la fecha actual	Trigger	Stock suficiente; fecha actual; estados
Cálculo que se usa en varios lugares	Función	Total con descuento
Operación de varios pasos que debe ser todo o nada	Procedimiento en una transacción	Procesar una transferencia
Formato, experiencia de usuario y autorización	API y front	Mensajes claros, validación previa, permisos

> **Atención.** No usar `CURRENT_DATE` ni otras funciones no determinísticas dentro de un `CHECK`: el resultado cambia con el tiempo y se rompe al restaurar un backup. Las reglas que dependen de la fecha actual van en un trigger.

Pasos

1. Revisar el DDL de la etapa 1 y completar las restricciones faltantes en `002_constraints.sql` (UNIQUE, CHECK, DEFAULT y acciones referenciales). Si hay tipos nuevos, crearlos antes de las tablas que los usan.
2. Implementar las **funciones** en `003_funciones.sql`. Cada una con un comentario que indique su propósito, sus parámetros y si es `STABLE` o `IMMUTABLE`.
3. Implementar los **procedimientos** transaccionales en `004_procedimientos.sql`: validan el estado, bloquean las filas necesarias con `FOR UPDATE`, modifican los stocks y cambian el estado; si algo falla, todo se revierte.

4. Implementar los **triggers** en `005_triggers.sql`. Encabezar cada uno con un comentario que cite la regla de negocio que protege (por ejemplo, RN-09).
5. Crear las **vistas** en `006_vistas.sql`.
6. Crear los **roles y privilegios** en `007_roles.sql`: quitar los privilegios públicos y otorgar a cada rol solo lo que necesita. La API se conectará con el rol de lectura/escritura, nunca con el propietario.
7. Escribir un **script de pruebas SQL** (`tests/db_tests.sql`) con casos que deben funcionar y casos que deben fallar. Cada regla de negocio necesita al menos un caso; indicar en un comentario qué error se espera.
8. Documentar con capturas una **demostración de concurrencia**: dos sesiones `psql` intentan vender la última botella de un vino al mismo tiempo. Mostrar qué pasa con y sin bloqueo (`SELECT ... FOR UPDATE`) y explicar el nivel de aislamiento utilizado.

Entregables

- Archivos `002_constraints.sql` a `007_roles.sql` y `tests/db_tests.sql`.
- Tabla de trazabilidad regla de negocio → mecanismo → caso de prueba.
- Informe corto de la demostración de concurrencia, con capturas.

Criterios de aceptación

- ☐ Se alcanzan todos los mínimos de la tabla de requisitos.
- ☐ Cada regla RN-01 a RN-15 tiene su mecanismo y su caso de prueba.
- ☐ Los casos que deben fallar fallan con un mensaje de error claro; los que deben funcionar, funcionan.
- ☐ Procesar una transferencia sin stock suficiente no deja ningún cambio a medias.
- ☐ La API podrá conectarse con un rol que no es propietario y no tiene permisos de sobra.

Etapa 3. Carga masiva de datos

Objetivo: trabajar con volúmenes parecidos a los de un sistema real. **Todas las tablas deben tener al menos una fila, al menos tres tablas deben superar las 10.000 filas y al menos una debe superar las 100.000** (se recomienda que los renglones de comprobante lleguen a 500.000).

Volúmenes recomendados

Tablas	Filas recomendadas
Localidad, cepa, maridaje, bodega, transportista, transportista cubre localidad	10 a 60 (catálogos reales)
Sala de venta, finca, empleado	15 a 150
Parcela, vino, composición, vino-maridaje, stock de depósito	300 a 1.000
Ingreso, transferencia, stock de sala	500 a 5.000
Renglón de ingreso, renglón de transferencia, lote de uva, análisis de calidad	3.000 a 15.000
Cliente	20.000
Comprobante	150.000
Renglón de comprobante	500.000 o más

Pasos

1. Cargar los **catálogos reales** (cepas, maridajes, departamentos y localidades de Mendoza, tipos de vino) desde archivos CSV. Los datos de catálogo no se generan al azar. En el anexo B hay una propuesta.
2. Definir el **orden de carga** según las dependencias de las claves foráneas (primero las tablas que no dependen de ninguna, después las que dependen de ellas). Dibujarlo en el informe.
3. Escribir el **script de carga en Python** con `psycopg`, `random` y `Faker`. Fijar una **semilla** para que dos ejecuciones produzcan los mismos datos.
4. Generar datos **coherentes**: fechas lógicas (la cosecha antes de la elaboración, el ingreso antes de la venta), porcentajes de corte que suman 100, precios y descuentos consistentes con el tipo de cliente, y stocks que nunca sean negativos.
5. Resolver cómo cargar el historial **sin chocar con los triggers** (por ejemplo, deshabilitarlos durante la carga histórica con `ALTER TABLE ... DISABLE TRIGGER`, volver a habilitarlos y validar el resultado con consultas de control). Documentar la decisión y sus riesgos.
6. Hacer la carga de forma **eficiente**: por lotes con `execute_values`/`executemany` o con `COPY`. Medir y registrar el tiempo de una inserción fila por fila frente a una por lotes.
7. Validar con **consultas de control**: conteo por tabla, filas huérfanas, stock negativo, comprobantes cuyo total no coincide con la suma de sus renglones.
8. Generar un volcado de la base cargada con `pg_dump` (`dump_completo.sql.gz`) para poder restaurarla.

Entregables

- Carpeta `db/seed/` con los CSV de catálogo y el script de carga, y un README con los pasos para ejecutarlo.
- Reporte con el conteo de filas por tabla, los tiempos de carga y las consultas de control con sus resultados.
- Volcado `dump_completo.sql.gz` (o instrucciones para generarlo si supera el tamaño permitido).

Criterios de aceptación

- ☐ Se cumplen los mínimos de volumen.
- ☐ Dos ejecuciones con la misma semilla producen los mismos datos.
- ☐ Las consultas de control no encuentran inconsistencias.
- ☐ El informe compara la carga fila por fila con la carga por lotes.
- ☐ Está documentado cómo se manejaron los triggers durante la carga.

Etapa 4. Índices y rendimiento

Objetivo: diseñar índices con evidencia medida y entender su costo. Con más de 100.000 filas las diferencias son visibles.

Requisitos mínimos

- Al menos **5 índices** creados por el grupo en `008_indices.sql`, que incluyan: uno **compuesto**, uno **parcial** (con `WHERE`), uno sobre una **clave foránea** (PostgreSQL no indexa las claves foráneas

automáticamente) y uno de **otro tipo o variante** (de expresión, BRIN o con `INCLUDE`), cada uno justificado.

- Al menos **3 consultas** medidas con `EXPLAIN (ANALYZE, BUFFERS)` antes y después de crear los índices.

Pasos

1. Elegir al menos tres consultas candidatas de la sección 8 que involucren muchas filas (por ejemplo S1, S4, S8 y S10).
2. Ejecutar `ANALYZE` y medir cada consulta con `EXPLAIN (ANALYZE, BUFFERS)` **solo con los índices de claves primarias y UNIQUE**. Guardar el plan y el tiempo.
3. Diseñar los índices a partir de lo que muestran los planes (recorridos secuenciales sobre tablas grandes, ordenamientos costosos, joins sin índice).
4. Crear los índices en `008_indices.sql` y volver a ejecutar `ANALYZE`.
5. Repetir las mediciones y armar una **tabla comparativa** con tiempo antes, tiempo después, tipo de recorrido (Seq Scan, Index Scan, Bitmap Scan) y filas leídas.
6. Justificar el **orden de las columnas** del índice compuesto y mostrar un índice que evaluaron y **descartaron**, explicando por qué.
7. Medir el **costo de escritura**: cuánto tarda la carga masiva de una tabla grande con y sin los índices.
8. Mostrar un caso en que el planificador **no** usa un índice existente y explicar la causa (por ejemplo, baja selectividad o estadísticas desactualizadas).

Entregables y criterios de aceptación

- `db/migrations/008_indices.sql` y el informe de rendimiento con los planes completos en un anexo.

☐ Hay al menos 5 índices, incluyendo los tipos pedidos, y todos tienen justificación.

☐ La tabla comparativa muestra una mejora medida en al menos 3 consultas.

☐ Se explica por qué algún índice se descartó y cuál es el costo de escritura de los índices creados.

Etapa 5. Consultas SQL

Objetivo: resolver en SQL las operaciones de inserción, modificación, borrado y consulta listadas en la sección 8, sobre la base ya cargada y con sus reglas activas.

Pasos

1. Crear cuatro archivos: `consultas/update.sql`, `consultas/insert.sql`, `consultas/delete.sql` y `consultas/select.sql`.
2. Para cada consigna escribir un encabezado con su código (por ejemplo U3), el enunciado, la consulta y una breve explicación de la estrategia elegida (joins, subconsultas, CTE, funciones de ventana, etc.).
3. Probar cada sentencia que modifica datos (UPDATE, INSERT, DELETE) **dentro de una transacción**: `BEGIN;` → consulta de control previa → sentencia → consulta de control posterior → `ROLLBACK;`. Documentar cuántas filas se vieron afectadas.
4. Cuando una consigna tiene parámetros (un nombre de bodega, una sala, una localidad), usar valores concretos del dataset y dejar indicado dónde se cambian.
5. Observar si alguna sentencia dispara triggers o choca con constraints. Anotar qué pasó y cómo se resolvió (por ejemplo, el borrado de comprobantes arrastra sus renglones por `ON DELETE CASCADE`).

6. Ejecutar `EXPLAIN (ANALYZE)` en las consultas que se van a usar para la etapa 4 y en las consultas avanzadas S9 a S12.

Criterios de aceptación

- ☐ Están resueltas todas las consignas de la sección 8, cada una con su explicación.
- ☐ Las sentencias de modificación muestran estado antes y después y se probaron en transacción.
- ☐ Las consultas avanzadas usan efectivamente CTE, funciones de ventana, subconsultas correlacionadas o `NOT EXISTS` según corresponda.

Etapa 6. API REST

Objetivo: exponer la base de datos mediante una API, sin ocultar el SQL y sin duplicar las reglas que ya viven en la base.

Requisitos

- Acceso a la base con **driver nativo y consultas parametrizadas** (nunca concatenar texto del usuario dentro del SQL).
- Una **capa de acceso a datos propia**: un módulo con una función por consulta, para que el SQL no quede desparramado por los controladores.
- **Pool de conexiones** y manejo explícito de transacciones (`BEGIN`, `COMMIT`, `ROLLBACK`) cuando una operación involucra más de una sentencia.
- Conexión con el **rol de lectura/escritura** creado en la etapa 2. Credenciales en variables de entorno y un archivo `.env.example` sin secretos.
- **Paginación** en los listados y **filtros** en el catálogo de vinos (por tipo, cepa, bodega, añada).
- **Traducción de errores de PostgreSQL a códigos HTTP** usando el código SQLSTATE (ver tabla).
- **Documentación** de la API con OpenAPI/Swagger o una colección Postman.
- Al menos **8 pruebas automáticas**, de las cuales al menos 3 deben provocar la violación de una regla de la base y verificar el código de respuesta.

Endpoints mínimos

Recurso	Método y ruta	Descripción
Vinos	GET /vinos, GET /vinos/{cod}	Catálogo con filtros y paginación; detalle con cepas, porcentajes y maridajes
Vinos	POST /vinos, PUT /vinos/{cod}, DELETE /vinos/{cod}	Alta con composición, modificación (audita el precio) y baja
Clientes	GET, POST, PUT, DELETE sobre /clientes	CRUD contemplando los tres tipos
Comprobantes	POST /comprobantes, GET /comprobantes/{nro}	La venta completa (cabecera y renglones) en una sola transacción
Transferencias	POST /transferencias, POST /transferencias/{nro}/renglones	Alta y carga de renglones mientras está en confección
Transferencias	POST /transferencias/{nro}/terminar, POST /transferencias/{nro}/procesar	Cambio de estado; procesar invoca el procedimiento
Ingresos	POST /ingresos, POST /ingresos/{cod}/renglones, POST /ingresos/{cod}/procesar	Mismo circuito que las transferencias

Recurso	Método y ruta	Descripción
Reportes	GET /reportes/top-vinos, /ventas-por-sala, /vinos-sin-stock, /transportista-activo, /trazabilidad/{cod}	Consultas de la sección 8 expuestas como endpoints
Salud	GET /salud	Verifica que la API llega a la base

Traducción de errores

Situación en PostgreSQL	SQLSTATE	Respuesta HTTP sugerida
UNIQUE violado	23505	409 Conflict
FOREIGN KEY violada (dato relacionado inexistente o con dependientes)	23503	409 Conflict o 422
CHECK violado	23514	422 Unprocessable Entity
NOT NULL violado	23502	422 Unprocessable Entity
Regla de negocio rechazada por un trigger o procedimiento (`RAISE EXCEPTION`)	P0001	409 Conflict, con el mensaje de la regla
Texto con formato inválido para el tipo	22P02	400 Bad Request
Fallo de serialización o deadlock	40001 / 40P01	503 Service Unavailable, con reintento sugerido

Pasos

1. Elegir lenguaje y framework, y crear la estructura del proyecto dentro de `api/`.
2. Implementar la conexión con pool, leyendo la configuración desde variables de entorno, y el endpoint `/salud`.
3. Escribir la capa de acceso a datos: una función por consulta, todas parametrizadas.
4. Implementar el catálogo de vinos con filtros y paginación, y luego los demás CRUD.
5. Implementar las operaciones transaccionales (comprobantes, transferencias e ingresos) llamando a las funciones y procedimientos de la base. **La API no recalcula totales ni stock.**
6. Implementar el manejo central de errores con la tabla de traducción de SQLSTATE.
7. Exponer los reportes de la sección 8 como endpoints de solo lectura.
8. Escribir las pruebas automáticas, incluyendo las que provocan errores a propósito (stock insuficiente, cambio de estado inválido, precio negativo).
9. Demostrar una **inyección SQL** fallida: enviar un valor malicioso a un endpoint con filtro y mostrar que, por estar parametrizada, se trata como un simple texto. Dejar la captura en el informe.
10. Escribir el `docker-compose.yml` que levante base y API.

Criterios de aceptación

- ☐ Un solo comando (`docker compose up`) levanta base y API.
- ☐ Toda consulta está parametrizada y la API usa el rol sin privilegios de propietario.
- ☐ Un intento de vender sin stock devuelve un error claro (409) y no deja datos a medias.
- ☐ Las pruebas automáticas pasan y cubren errores de reglas de la base.

☐ La documentación de la API está disponible y es coherente con las rutas reales.

Etapa 7. Front

Objetivo: ver cómo el dato viaja desde la base hasta la pantalla y cómo las reglas de la base se perciben desde el usuario. El foco no es el diseño visual.

Pantallas mínimas

Pantalla	Qué debe permitir
Catálogo de vinos	Listado paginado con filtros por tipo, cepa, bodega y añada; detalle con composición y maridajes
Alta y edición de vino	Formulario que incluye la composición de cepas; debe mostrar el error cuando los porcentajes no suman 100
Nueva venta	Elegir cliente y sala, agregar vinos y ver el total **calculado por el backend**; mostrar el error de stock insuficiente
Transferencias	Crear una transferencia, cargar renglones y pasar de estado con botones que se habilitan según el estado actual
Tablero	Ranking de los 10 vinos más vendidos, ventas por sala en el último trimestre (gráfico) y vinos sin stock en una sala elegida

Pasos

1. Dibujar un boceto simple de cada pantalla (puede ser a mano).
2. Armar la estructura del front en `front/` y una capa que concentre las llamadas a la API, con la URL base configurable.
3. Implementar cada pantalla empezando por el catálogo y terminando por el tablero.
4. Mostrar al usuario los **mensajes de error de las reglas de negocio** tal como los devuelve la API, sin ocultarlos ni reemplazarlos por un mensaje genérico.
5. Verificar que el front **no calcula** totales, descuentos ni stock.
6. Agregar el front al `docker-compose.yml`.

Criterios de aceptación

☐ Las cinco pantallas funcionan contra la API real.

☐ Una venta sin stock muestra el mensaje de la regla y no deja datos a medias.

☐ El front no contiene lógica de cálculo de importes ni de stock.

Etapa 8. Despliegue en la nube

Objetivo: que el sistema funcione en un servidor que no es la computadora del grupo, con las medidas básicas de seguridad y operación que se esperan de cualquier sistema real.

Requisitos

- **Base de datos PostgreSQL** desplegada en la nube (servicio gestionado o contenedor en un servidor virtual).
- **API y front** desplegados con una **URL pública** con HTTPS.

- **Migraciones aplicadas** en la nube desde los archivos SQL del repositorio, en orden.
- Conexión a la base **cifrada** (`sslmode=require` o equivalente) y puerto de la base **no expuesto** a todo internet (restringirlo por IP o a la red interna del proveedor).
- **Secretos** (contraseñas, cadenas de conexión) en variables de entorno del proveedor y nunca en el repositorio.
- La API usa el rol de **privilegios mínimos**, no el propietario.
- **Backup y restauración probados**: generar un backup con `pg_dump`, restaurarlo en una base nueva con `pg_restore` o `psql` y verificar los conteos.
- Datos cargados en la nube: si el plan gratuito limita el almacenamiento, se acepta una carga reducida (al menos 10.000 filas en una tabla) **con justificación**, manteniendo el volumen completo en la versión local.

Sobre los proveedores. Cada grupo elige dónde desplegar. Algunos ejemplos habituales son Neon, Supabase, Render, Railway, Fly.io, Aiven, los planes educativos o de capa gratuita de AWS, Azure y Google Cloud, o un servidor virtual propio con Docker.
Los planes gratuitos y sus límites cambian seguido. Antes de elegir, hay que verificar en el sitio del proveedor las condiciones vigentes (almacenamiento, tiempo de inactividad, conexiones y si pide tarjeta).

Pasos

1. Comparar al menos dos proveedores y elegir uno. Anotar la decisión y los límites del plan en el informe.
2. Crear la base en la nube y los tres roles de la etapa 2 (propietario, lectura/escritura de la API y solo lectura).
3. Aplicar las migraciones de `db/migrations/` en orden con el rol propietario.
4. Cargar los datos (completos o reducidos, según el plan) y ejecutar las consultas de control de la etapa 3.
5. Configurar la API con variables de entorno y desplegarla. Verificar `/salud`.
6. Desplegar el front apuntando a la URL pública de la API. Revisar que la API permita ese origen (CORS).
7. Restringir el acceso a la base (IP permitidas o red interna) y comprobar que una conexión no autorizada es rechazada.
8. Probar **backup y restauración** y documentar los comandos y los resultados.
9. Repetir en la nube un caso que dispare un trigger (por ejemplo, una venta sin stock) y dejar la captura.
10. Completar el `README` con la URL pública y los pasos para reproducir el despliegue.

Criterios de aceptación

- ☐ Cualquier persona puede abrir la URL pública y usar el sistema.
- ☐ La conexión a la base está cifrada y el puerto no está abierto a todo internet.
- ☐ No hay secretos en el repositorio (incluido su historial).
- ☐ El backup se restauró en una base nueva y los conteos coinciden.
- ☐ Una regla de la base (por ejemplo, stock insuficiente) se comprobó funcionando en la nube.

Etapa 9. Documentación y defensa

Objetivo: dejar el proyecto reproducible por un tercero y demostrar que cada integrante lo comprende.

Pasos

1. Escribir el `README` principal: descripción, requisitos, cómo levantar todo en local con Docker Compose, cómo ejecutar migraciones, carga y pruebas, y la URL de la versión en la nube.
2. Armar el **informe final** (estructura en la sección 10) juntando los informes parciales de cada etapa.
3. Completar la tabla de trazabilidad regla de negocio → mecanismo → caso de prueba.
4. Preparar la **demo en vivo** sobre la versión en la nube (10 a 15 minutos): recorrer el flujo de venta, mostrar el rechazo de una venta sin stock, procesar una transferencia y mostrar una consulta con su plan de ejecución.
5. Preparar la **defensa oral individual**: cada integrante responde preguntas sobre cualquier parte del trabajo (ver ejemplos en el anexo C) y puede ser convocado a modificar algo en vivo.

8. Consultas SQL requeridas

Todas las consultas deben resolverse en SQL de PostgreSQL y estar comentadas. Los valores concretos (nombres de bodega, sala, localidad, vino) se eligen del dataset cargado. Cuando la consigna dice «la última semana» o «el último trimestre» debe calcularse a partir de la fecha actual (`CURRENT_DATE`), no con fechas fijas.

8.1 Modificación de datos (UPDATE)

Cód.	Consigna
U1	Recalcular el total de todos los comprobantes aplicando el descuento que corresponde al tipo de cliente.
U2	Aumentar un 20 % el salario de los empleados con más de 5 años de antigüedad.
U3	Incrementar un 13 % el precio de los vinos elaborados por una bodega determinada.
U4	En las salas cuyo día de cierre es lunes cambiarlo a miércoles y viceversa, en una sola sentencia.
U5	Aplicar un 15 % de descuento de precio a los 10 vinos menos vendidos de toda la red.
U6	A los exportadores con más de 3 permisos de exportación, subirles el descuento al 35 %.

8.2 Inserción de datos (INSERT)

Cód.	Consigna
I1	Insertar en la relación transportista-localidad una nueva cobertura que vincule la localidad de General Alvear con el transportista que menos envíos (ingresos trasladados) haya realizado en el último trimestre.
I2	Si existe algún vino tinto con graduación de 14° o más, crear el maridaje «cordero al disco» y asociarlo a todos los vinos que cumplan esa condición.
I3	Crear un nuevo comprobante con tres renglones para un cliente dado, en la sala con ID 1, dentro de una transacción y usando las funciones de cálculo.
I4	Insertar una nueva bodega.
I5	Insertar una nueva sala de venta.
I6	Crear un ingreso en confección de 120 botellas de un vino determinado (por ejemplo «Malbec Reserva» de 750 ml), elaborado por una bodega dada y trasladado por un transportista dado.

8.3 Borrado de datos (DELETE)

Cód.	Consigna
D1	Borrar los comprobantes con más de 5 años de antigüedad asociados a clientes socios del club.
D2	Eliminar las cepas que no componen ningún vino ni están plantadas en ninguna parcela.
D3	La sala de una localidad determinada fue dada de baja: eliminar los empleados de esa sala que trabajaron menos de dos años.
D4	Borrar los clientes socios que nunca compraron y se dieron de alta hace más de 5 años.
D5	Eliminar los ingresos en estado procesado de hace más de 365 días.

Cód.	Consigna
D6	Borrar los clientes socios sin compras que viven en una localidad específica.

8.4 Consultas (SELECT)

Cód.	Consigna
S1	Ranking de los 10 vinos con mayor cantidad de botellas vendidas en toda la red de salas.
S2	Listar códigos y nombres de los vinos que se vendieron en **todas** las salas (división relacional).
S3	Cantidad de clientes de cada tipo que compraron en una sala determinada en la última semana.
S4	Top 5 de las salas que solicitaron mayor cantidad de un vino determinado mediante transferencias en los últimos 15 días.
S5	Listado de las salas con menor cantidad de transferencias pendientes (no procesadas).
S6	Para una sala determinada, los vinos sin stock en la sala pero con stock en el depósito central.
S7	¿Cuál es la empresa de transporte con mayor actividad en el último mes?
S8	Monto total de ventas por sala en el último trimestre, en orden descendente.

8.5 Consultas avanzadas (SELECT)

Cód.	Consigna	Herramienta esperada
S9	Ficha de un vino: cepas con sus porcentajes, bodega elaboradora, maridajes y stock total (depósito más salas).	Joins, agregación y vista
S10	Rendimiento en kg por hectárea de cada parcela y campaña, e identificar las parcelas que rinden menos que el promedio de su finca.	CTE y subconsulta correlacionada
S11	Ranking de vinos dentro de cada sala por botellas vendidas y ventas acumuladas mensuales por sala.	`RANK() OVER (PARTITION BY ...)` y suma acumulada con ventana
S12	Clientes que compraron todos los vinos de una bodega determinada, y clientes que nunca compraron ningún vino tinto.	`NOT EXISTS` (división) y anti-join

9. Rúbrica de evaluación

El puntaje total es de 100 puntos. **La capa de datos (modelado, constraints, lógica, índices, carga y consultas) representa el 70 %** de la nota.

Criterio	Puntos	Qué se valora
Modelado (etapa 1)	15	DER correcto, esquema relacional en 3FN, justificación de decisiones, diccionario y DDL que corre desde cero
Constraints (etapa 2)	8	Cobertura de PK, FK, NOT NULL, UNIQUE, CHECK y DEFAULT; acciones referenciales justificadas; ENUM y DOMAIN
Funciones, procedimientos, triggers y transacciones (etapa 2)	17	Corrección, cobertura de las reglas RN-01 a RN-15, casos de prueba, manejo de concurrencia, vistas y roles
Carga masiva (etapa 3)	8	Volúmenes, coherencia de los datos, reproducibilidad, eficiencia y consultas de control
Índices y rendimiento (etapa 4)	10	Evidencia con EXPLAIN, variedad y justificación de índices, costo de escritura, caso en que no se usa un índice
Consultas SQL (etapa 5)	12	Resolución correcta, prueba en transacción, uso adecuado de subconsultas, CTE, ventanas y división
API (etapa 6)	10	SQL parametrizado, capa de acceso a datos, transacciones, traducción de errores, pruebas y documentación
Front (etapa 7)	5	Pantallas mínimas funcionando, errores de reglas visibles, sin lógica de cálculo en el cliente
Nube y seguridad (etapa 8)	10	Despliegue, HTTPS, conexión cifrada, acceso restringido, secretos, roles, backup y restauración probados
Documentación, Git y demo (etapa 9)	5	README reproducible, informe, historial de commits de todos y demo en vivo
Total	**100**	

Nota individual

La nota grupal se pondera con la **defensa oral individual**. [Completar: coeficiente o mecanismo que defina la cátedra.] Un integrante que no pueda explicar el modelo ni las reglas de la base puede ver reducida su nota aunque el trabajo grupal sea excelente.

Situaciones que descuentan puntos

- Uso de ORM o de herramientas que generan el esquema o el SQL sin que el grupo lo haya escrito.
- Secretos (contraseñas, cadenas de conexión) publicados en el repositorio, incluso en el historial.
- La base no se puede crear desde cero con los archivos entregados.
- La API se conecta con el usuario propietario o con un superusuario.
- Totales, descuentos o stock calculados en la API o en el front en lugar de la base.
- Historial de Git con un solo integrante.

10. Estructura del repositorio y entrega final

10.1 Estructura sugerida del repositorio

```
/
|-- README.md
|-- docker-compose.yml
|-- .env.example
|-- db/
|   |-- migrations/
|   |   |-- 001_esquema.sql
|   |   |-- 002_constraints.sql
|   |   |-- 003_funciones.sql
|   |   |-- 004_procedimientos.sql
|   |   |-- 005_triggers.sql
|   |   |-- 006_vistas.sql
|   |   |-- 007_roles.sql
|   |   `-- 008_indices.sql
|   |-- seed/            (CSV de catalogo y script de carga)
|   `-- tests/           (db_tests.sql)
|-- consultas/           (insert.sql, update.sql, delete.sql, select.sql)
|-- api/
|-- front/
`-- docs/                (DER, esquema relacional, diccionario, informes)
```

10.2 Estructura del informe final

1. Introducción y alcance.
2. Modelo de datos: DER, esquema relacional, normalización, diccionario de datos y decisiones de diseño.
3. Integridad y lógica en la base: tabla de trazabilidad regla → mecanismo → prueba, guía de «dónde va cada regla» y demostración de concurrencia.
4. Carga masiva: catálogos, orden de carga, estrategia frente a los triggers, tiempos y consultas de control.
5. Índices y rendimiento: tabla comparativa antes y después, planes de ejecución y costo de escritura.
6. Consultas SQL: enlace a los archivos y comentarios sobre las más interesantes.
7. API y front: arquitectura, endpoints, traducción de errores, pruebas y demostración de inyección SQL fallida.
8. Despliegue en la nube: proveedor elegido y por qué, configuración de seguridad, backup y restauración.
9. Conclusiones: dificultades, decisiones que cambiarían y aprendizajes.

10.3 Lista de control de la entrega final

- ☐ El repositorio está en GitHub o GitLab con commits de todos los integrantes y sin secretos.
- ☐ `docker compose up` levanta base, API y front en una máquina limpia.
- ☐ Los archivos de migración se aplican en orden sobre una base vacía sin errores.
- ☐ Se cumplen los mínimos de constraints, funciones, procedimientos, triggers, vistas e índices.
- ☐ Hay más de 100.000 filas en al menos una tabla y el script de carga es reproducible.
- ☐ Las consultas de la sección 8 están resueltas, comentadas y probadas.

- ☐ La versión en la nube está activa con URL pública y HTTPS.
- ☐ El backup se restauró en una base nueva.
- ☐ El informe final y el README están completos.
- ☐ Cada integrante preparó su defensa oral.

Anexo A. Ejemplos en PostgreSQL

Los siguientes fragmentos son **modelos orientativos** para mostrar la sintaxis y el estilo esperado. No cubren todo lo que pide el enunciado y cada grupo debe adaptarlos a su propio modelo, nombres y decisiones.

A.1 Tipo enumerado y dominio

```
CREATE TYPE estado_documento AS ENUM ('en_confeccion', 'terminado', 'procesado');

CREATE DOMAIN cuit_dom AS CHAR(11)
  CHECK (VALUE ~ '^[0-9]{11}$');
```

A.2 Tabla con constraints

```
CREATE TABLE vino (
  cod_vino    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre      VARCHAR(80)   NOT NULL,
  tipo        VARCHAR(12)   NOT NULL
              CHECK (tipo IN ('tinto', 'blanco', 'rosado', 'espumante')),
  anada       SMALLINT      NOT NULL CHECK (anada BETWEEN 1990 AND 2100),
  volumen_ml  INTEGER       NOT NULL CHECK (volumen_ml IN (187, 375, 750, 1500, 3000)),
  graduacion  NUMERIC(4,2)  NOT NULL CHECK (graduacion BETWEEN 5 AND 20),
  precio      NUMERIC(10,2) NOT NULL CHECK (precio > 0),
  cuit_bodega cuit_dom      NOT NULL
              REFERENCES bodega (cuit) ON DELETE RESTRICT ON UPDATE CASCADE,
  UNIQUE (nombre, anada, volumen_ml)
);

CREATE TABLE stock_sala (
  id_sala  INTEGER NOT NULL REFERENCES sala_venta (id_sala) ON DELETE RESTRICT,
  cod_vino INTEGER NOT NULL REFERENCES vino (cod_vino)      ON DELETE RESTRICT,
  cantidad INTEGER NOT NULL DEFAULT 0 CHECK (cantidad >= 0),
  PRIMARY KEY (id_sala, cod_vino)
);
```

A.3 Índice parcial y índice compuesto

```
-- Solo indexa lo que todavia esta pendiente (tabla chica dentro de una grande)
CREATE INDEX idx_transferencia_pendientes
  ON transferencia (id_sala, fecha_confeccion)
  WHERE estado <> 'procesado';

-- Ventas por sala y rango de fechas
CREATE INDEX idx_comprobante_sala_fecha
  ON comprobante (id_sala, fecha);
```

A.4 Función

```
CREATE OR REPLACE FUNCTION fn_stock_sala(p_sala INTEGER, p_vino INTEGER)
RETURNS INTEGER
LANGUAGE sql STABLE AS $$
  SELECT COALESCE(
    (SELECT cantidad FROM stock_sala
      WHERE id_sala = p_sala AND cod_vino = p_vino), 0);
$$;
```

A.5 Trigger BEFORE INSERT que valida stock (RN-09)

```
CREATE OR REPLACE FUNCTION trg_validar_stock_venta() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
  v_sala INTEGER;
BEGIN
  SELECT id_sala INTO v_sala FROM comprobante WHERE nro = NEW.nro_comprobante;
  IF fn_stock_sala(v_sala, NEW.cod_vino) < NEW.cantidad THEN
    RAISE EXCEPTION 'Stock insuficiente del vino % en la sala % (RN-09)',
                    NEW.cod_vino, v_sala;
  END IF;
  RETURN NEW;
END;
$$;

CREATE TRIGGER trg_renglon_valida_stock
BEFORE INSERT ON renglon_comprobante
FOR EACH ROW EXECUTE FUNCTION trg_validar_stock_venta();
```

A.6 Procedimiento transaccional (RN-11)

```
CREATE OR REPLACE PROCEDURE sp_procesar_transferencia(p_nro INTEGER)
LANGUAGE plpgsql AS $$
DECLARE
  v_estado estado_documento;
BEGIN
  SELECT estado INTO v_estado
    FROM transferencia WHERE nro = p_nro FOR UPDATE;

  IF v_estado IS DISTINCT FROM 'terminado' THEN
    RAISE EXCEPTION 'La transferencia % no existe o no esta terminada (RN-11)', p_nro;
  END IF;

  -- Descuenta del deposito central. Si falta stock, el CHECK (cantidad >= 0)
  -- hace fallar la sentencia y se revierte todo el procedimiento.
  UPDATE stock_deposito d
     SET cantidad = d.cantidad - r.cantidad
    FROM renglon_transferencia r
   WHERE r.nro = p_nro AND d.cod_vino = r.cod_vino;

  -- Suma al stock de la sala (crea la fila si todavia no existia)
  INSERT INTO stock_sala (id_sala, cod_vino, cantidad)
  SELECT t.id_sala, r.cod_vino, r.cantidad
    FROM transferencia t JOIN renglon_transferencia r ON r.nro = t.nro
   WHERE t.nro = p_nro
  ON CONFLICT (id_sala, cod_vino)
  DO UPDATE SET cantidad = stock_sala.cantidad + EXCLUDED.cantidad;

  UPDATE transferencia SET estado = 'procesado' WHERE nro = p_nro;
END;
$$;

-- Uso:  CALL sp_procesar_transferencia(1234);
```

Para pensar. En este ejemplo, si un vino del renglón no tiene fila en `stock_deposito`, el `UPDATE` no descuenta nada y no da error. Cada grupo debe detectar ese caso y decidir cómo manejarlo (por ejemplo, verificando antes que exista y que alcance).

A.7 Roles con privilegios mínimos

```
-- La contrasena real NO va en el repositorio: se pasa por variable de entorno.
CREATE ROLE app_rw LOGIN PASSWORD 'cambiar_por_un_secreto';

REVOKE ALL ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO app_rw;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_rw;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO app_rw;
GRANT EXECUTE ON ALL ROUTINES IN SCHEMA public TO app_rw;
```

A.8 Demostración de concurrencia: la última botella

Ejecutar en dos terminales `psql` distintas, suponiendo que el stock de la sala 1 para el vino 10 es de una sola botella.

Paso	Sesión A	Sesión B
1	`BEGIN;`	`BEGIN;`
2	`SELECT cantidad FROM stock_sala WHERE id_sala = 1 AND cod_vino = 10 FOR UPDATE;` (devuelve 1)	
3		La misma consulta con `FOR UPDATE`: **queda esperando** a que A termine
4	`UPDATE stock_sala SET cantidad = cantidad - 1 WHERE id_sala = 1 AND cod_vino = 10;` y luego `COMMIT;`	
5		Recién ahora obtiene el resultado, que ya es 0: la venta se rechaza

Repetir el experimento **sin** `FOR UPDATE` y explicar por qué, según el nivel de aislamiento, las dos sesiones pueden creer que hay stock y terminar vendiendo la misma botella dos veces.

Anexo B. Catálogos sugeridos para la carga

Estos catálogos se cargan desde archivos CSV y no se generan al azar. Los valores de temperatura son orientativos.

B.1 Cepas

Tipo	Cepas
Tintas	Malbec, Cabernet Sauvignon, Merlot, Syrah, Bonarda, Tempranillo, Pinot Noir
Blancas	Chardonnay, Sauvignon Blanc, Torrontés, Chenin Blanc, Semillón

B.2 Maridajes

Maridaje	Temperatura de servicio (°C)
Carnes rojas	17
Quesos duros	16
Pastas con salsa roja	16
Carnes blancas	12
Quesos blandos	10
Pescados y mariscos	8
Entradas y fiambres	10
Postres	8

B.3 Departamentos de Mendoza (para localidades y fincas)

San Rafael, General Alvear, Malargüe, Luján de Cuyo, Maipú, Tupungato, Tunuyán, San Carlos, Lavalle, Rivadavia, Junín, San Martín, Santa Rosa, La Paz, Godoy Cruz, Guaymallén, Las Heras y Capital.

B.4 Otros datos de referencia

- Presentaciones habituales: 187 ml, 375 ml, 750 ml, 1500 ml (magnum) y 3000 ml.
- Tipos de vino: tinto, blanco, rosado y espumante.
- Nombres de bodegas, fincas, clientes y transportistas: generados con Faker o inventados por el grupo (son ficticios).
- Campañas: los últimos 5 años, con cosecha entre febrero y abril.